

Задача A1. Техника добавления цифры: максимум

Формально, требуется добавить 1 цифру так, чтобы максимизировать полученное число. Очевидно, что выгодно ставить 9 в начало.

Задача A2. Техника добавления цифры: минимум

Формально, требуется добавить 1 цифру так, чтобы минимизировать полученное число. В отличие от простой версии, здесь не получится просто поставить 0 в начало – это запрещено по условию, ведь число не изменяется.

Заметим очевидное, но важное! После добавления цифры, длина числа всегда будет на 1 больше. Попробуем поставить число 1 в начало и подумаем когда это выгодно. Например, если первая цифра 2 то мы перейдем:

$$2\dots \rightarrow 12\dots$$

и это оптимально. Если вставлять не в начало, то в новом старшем разряде будет стоять 2, а это больше того что мы уже показали. Также работает для любой первой цифры большей 1.

Однако, если первая цифра 1, то переход

$$1\dots \rightarrow 11\dots$$

не оптимальный, ведь это тоже самое как если бы мы вставляли 1 между первой и второй цифрой. Но туда уже можно вставить 0 и это будет лучшим вариантом. Вот и все решение.

Задача В. Свой любимчик

Пусть сумма баллов участника i равна a_i . Отсортируем участников по убыванию суммы баллов. Пусть мы на месте k , тогда мы обязаны занизить баллы всем кто выше нашего места. Пусть участник $i \in [1, k)$ имеет баллы $b_1 \geq \dots \geq b_6$ по каждой из задач (т.е. максимальный балл среди посылок по данной задаче), упорядоченных по убыванию. Чтобы отменить как можно меньше задач участника, нам выгодно начинать с задачи с самым большим баллом. Таким образом, будет пытаться отменять до тех пор, пока не станет:

$$b_1 + \dots + b_j > a_k.$$

Такой алгоритм будет работать $O(n \log(n) + q)$, то есть столько элементарных операций умноженную на какую-то универсальную константу (обычно зависящей от реализации).

Задача С. Суперестановка

Любая суперестановка-23 имеет длину больше чем 400. Поэтому, при $n \geq 23$ массив a не может быть суперестановкой, и ответ всегда «NO».

Решение для $n \leq 23$. Пусть $dp[mask]$ — минимальный индекс i такой, что префикс длины i является суперестановкой для элементов лежащих в $mask$. Переход: если мы добавляем элемент j в маску, то обновляем состояние $dp[mask|2^j]$, как $dp[mask|2^j] = \min(dp[mask|2^j], next[dp[mask]][j])$, где $next[i][j]$ — первая позиция в массиве a после индекса i , на которой стоит число j .

Если dp от полной маски будет $\leq n$, то данный массив является суперестановкой, иначе нет. Для восстановления ответа для каждой $mask$ можно хранить элемент j , которую мы последним добавили в $mask$.

Задача D. Трансформация матрицы

Пусть заданы две целочисленные матрицы A и B размера $n \times m$, где $n, m \geq 1$. Нужно за не более чем $n \cdot m$ операций превратить A в B с помощью шагов вида:

$$\begin{aligned} A[x][y] &\leftarrow A[x][y] + c, \\ A[x][j] &\leftarrow A[x][j] - c, \\ A[i][y] &\leftarrow A[i][y] - c, \\ A[i][j] &\leftarrow A[i][j] + c, \end{aligned}$$

где $x \neq i$, $y \neq j$ и $1 \leq c \leq 2 \cdot 10^9$.

Покажем, что каждая такая операция не меняет:

- сумму элементов в любой строке;
- сумму элементов в любом столбце.

Действительно, для строки x происходит прибавление c в столбце y и вычитание c в столбце j , суммарное изменение = 0. Аналогично для строки i , столбца y и столбца j .

Из этого следует необходимое условие выполнимости $A \rightarrow B$:

$$\sum_{j=1}^m A[i][j] = \sum_{j=1}^m B[i][j] \quad (\text{для всех } i) \quad \text{и} \quad \sum_{i=1}^n A[i][j] = \sum_{i=1}^n B[i][j] \quad (\text{для всех } j).$$

Если где-то суммы строк или столбцов не совпадают, **ответ сразу**

−1.

Пусть теперь во всех строках и столбцах суммы в A и B совпадают. Покажем, что этого достаточно, чтобы перевести A в B не более чем за $n \times m$ операций.

Идея. Пройдёмся по всем элементам (i, j) в **первых** $(n - 1)$ строках и $(m - 1)$ столбцах. Для каждой клетки:

$$\text{diff} = B[i][j] - A[i][j].$$

Если $\text{diff} \neq 0$, применим **одну** операцию, которая исправит $A[i][j]$ на нужное значение (прибавит или вычит $|\text{diff}|$), а побочное изменение отразится только на ячейках последней строки ($i = n$) и последнего столбца ($j = m$). Например:

- если $\text{diff} > 0$, подойдёт операция

$$(x, y, i', j', c) = (i, j, n, m, \text{diff}),$$

которая увеличивает $A[i][j]$ на diff ;

- если $\text{diff} < 0$, делаем

$$(x, y, i', j', c) = (i, m, n, j, |\text{diff}|)$$

для уменьшения $A[i][j]$.

Так мы «починим» значения во всех клетках (i, j) при $1 \leq i \leq n - 1$, $1 \leq j \leq m - 1$. После этого совпадут $A[i][j]$ и $B[i][j]$ везде, кроме возможно последней строки $i = n$ и последнего столбца $j = m$. Но из-за совпадения сумм по строкам и столбцам (а операции их не меняют!) оставшиеся клетки автоматически становятся равными соответствующим элементам в B .

Число операций. Очевидно, каждую клетку (i, j) мы «чиним» не более одного раза. Итого максимум $(n - 1) \times (m - 1)$ операций, что не превышает $n \times m$.

Задача Е. Прибавление на отрезке

Для решения задачи будем хранить не сами множества $S_1, S_2, \dots, S_n$, а информацию о каждом значении x . А именно:

- **Отрезки присутствия** — это интервалы, где значение x содержится.
- **Отрезки отсутствия** — это интервалы, где значение x отсутствует.

Добавление значения x на отрезке $[l, r]$.

При добавлении числа x на отрезке $[l, r]$, нужно обновить отрезки присутствия и отсутствия. Рассмотрим обновление отрезков присутствия.

1. Найти все отрезки присутствия, которые пересекаются с $[l, r]$.

2. Объединить эти пересекающиеся отрезки с $[l, r]$ в один общий диапазон.
3. Удалить старые пересекающиеся отрезки из списка и добавить объединённый отрезок.

Например, если текущие отрезки присутствия для x равны $[2, 3]$, $[6, 8]$ и $[10, 12]$, а добавляемый отрезок $[5, 10]$, то после обновления отрезки станут $[2, 3]$ и $[5, 12]$.

Обновление отрезков отсутствия выполняется аналогично:

- Удаляем пересекающиеся с $[l, r]$ отрезки отсутствия.
- Если отрезки отсутствия частично пересекаются с $[l, r]$, разделяем их на части, не пересекающиеся с $[l, r]$.
- Обновляем список оставшимися частями отрезков.

Ответы на запросы.

- **Пересечение:** значение x входит в пересечение всех множеств на отрезке $[l, r]$, если существует отрезок присутствия, который полностью покрывает $[l, r]$.
- **Объединение:** значение x входит в объединение всех множеств на отрезке $[l, r]$, если не существует отрезка отсутствия, который полностью покрывает $[l, r]$.

Быстрый подсчёт отрезков.

Для проверки, покрывает ли отрезок $[l, r]$ заданный диапазон $[L, R]$, можно использовать следующую идею.

Каждый отрезок $[l, r]$ представим как точку (x, y) , где:

- $x = l$ — левая граница отрезка.
- $y = r$ — правая граница отрезка.

Задача сводится к подсчёту количества точек (x, y) , которые лежат в прямоугольной области: $0 \leq x \leq L$ и $R \leq y \leq n$

Для этого можно использовать двумерные структуры данных, например, Fenwick Tree или Segment Tree, которые позволяют эффективно работать с запросами на подсчёт точек в прямоугольниках и добавление новых точек.

Итоговое решение.

- Для добавления значения x на отрезке $[l, r]$:
 - Обновляем отрезки присутствия: объединяем пересекающиеся отрезки с $[l, r]$.
 - Обновляем отрезки отсутствия аналогично.
- Для пересечения считаем, количество отрезков присутствия которые покрывает $[l, r]$.
- Для объединения проверяем, n минус количество отрезков отсутствия которые покрывает $[l, r]$

Задача F. Сказочный город

Рассмотрим каждую компоненту связности графа отдельно.

Случай: количество ребер в компоненте чётно. Если количество рёбер чётное, то граф можно покрыть путями длины 2. Для этого:

1. Запустим обход в глубину (DFS) из произвольной вершины, поддерживая родителя.
2. На каждом шаге:
 - Собираем все обратные рёбра и «лишние» рёбра вниз.
 - Формируем пары рёбер для покрытия путями длины 2.
3. Если остаётся «лишнее» ребро, то объединим его в пару с ребром вверх, переданным от родительской вершины.
4. Если нет «лишнего» ребра, передаём вверх ребро, чтобы родительская вершина обработала его.

Так как количество рёбер чётное, всегда можно найти корректное разбиение. Для понимания алгоритма рекомендуется изучить тему DFS-tree.

Случай: количество ребер в компоненте нечётно. Если количество рёбер нечётное, возможны два случая: компонента является деревом или оно содержит цикл:

Случай: компонента содержит цикл.

1. Найдём цикл в компоненте. Пусть циклом будет последовательность вершин $c_1, c_2, \dots, c_k$ в порядке обхода.
2. Удалим одно ребро (c_1, c_k) . После этого оставшаяся компонента будет связной, а количество рёбер станет чётным. Применим описанный выше алгоритм для покрытия рёбер путями длины 2.
3. Теперь добавим удалённое ребро в один из путей:
 - Если существует путь, концом которого является вершина c_1 , добавим удалённое ребро в этот путь.
 - Если такого пути нет, рассмотрим путь, содержащий ребро (c_2, c_1) . Пусть это будет путь $[c_2, c_1, A]$. Перестроим этот путь в $[c_k, c_1, A]$, а удалённым ребром станет ребро (c_2, c_1) .
 - Повторим процесс для нового удалённого ребра. Для него мы будем рассматривать пути содержащие вершину c_2 .
 - Возможно новым удалённым ребром станет ребро (c_3, c_2) и для него рассмотрим пути содержащие вершину c_3 и так далее.
4. Если удалённым ребром станет ребро (c_k, c_{k-1}) , то просто добавим его в путь $[c_k, c_1, A]$.

Случай: компонента является деревом.

- Если у всех вершин нечётная степень, решения не существует. Подвесим дерево за произвольную вершину. Пусть у вершины, которая не является корнем, все дети будут листьями. Их количество будет чётным, и придется объединить их в пары между собой и удалить. Тогда эта вершина тоже станет листом. Это продолжится до тех пор, пока у корня не останется нечётное число детей-листьев, которые невозможно покрыть.
- Если существует вершина с чётной степенью, решение существует. Подвесим дерево за эту вершину. Алгоритм:
 1. Для каждого сына корня запустим алгоритм разбиения его поддерев на пути длины 2.

2. В поддереве каждого сына останется не более одного ребра. Получается из корня будут выходить пути длины 1 и 2.
3. Количество путей длины 1 будет нечетным. Количество путей длины 2 тоже будет нечетным.
4. Объединим один путь длины 1 и один путь длины 2, чтобы получить путь длины 3. После этого количество путей длины 1 станет чётным и их можно объединить в пары между собой, а пути длины 2 оставить как есть.